



Unidad 5 Clase 11

Acceso a estructura NoSQL

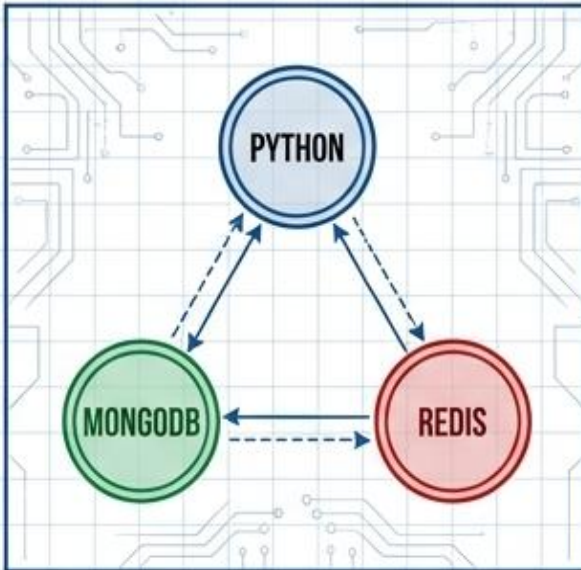
ARQUITECTURAS HÍBRIDAS Y PERSISTENCIA POLÍGLOTA

Diseñar soluciones híbridas que integren múltiples fuentes de datos relacionales y NoSQL, aplicando conceptos de persistencia políglota y evaluando su impacto en sistemas reales.

DE ACCEDER A BASES NOSQL A DISEÑAR ARQUITECTURAS HÍBRIDAS

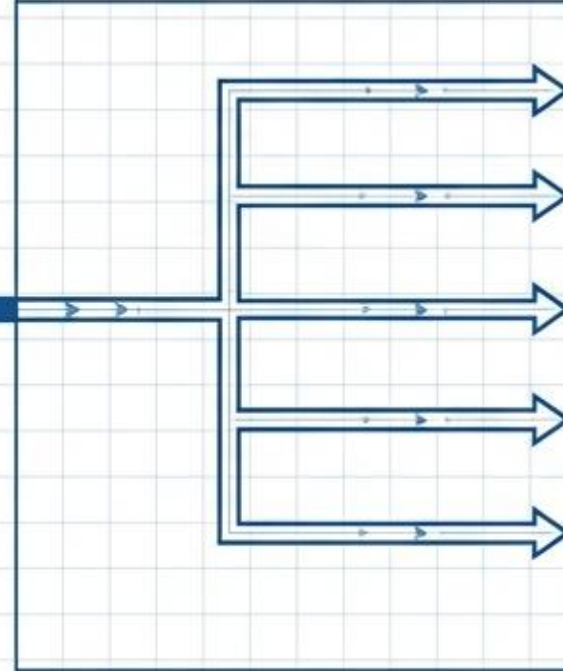
El foco deja de ser “cómo conecto una base” y pasa a ser “cómo diseño una solución donde cada motor tiene una responsabilidad”.

CLASE 10: PYTHON + MONGODB + REDIS

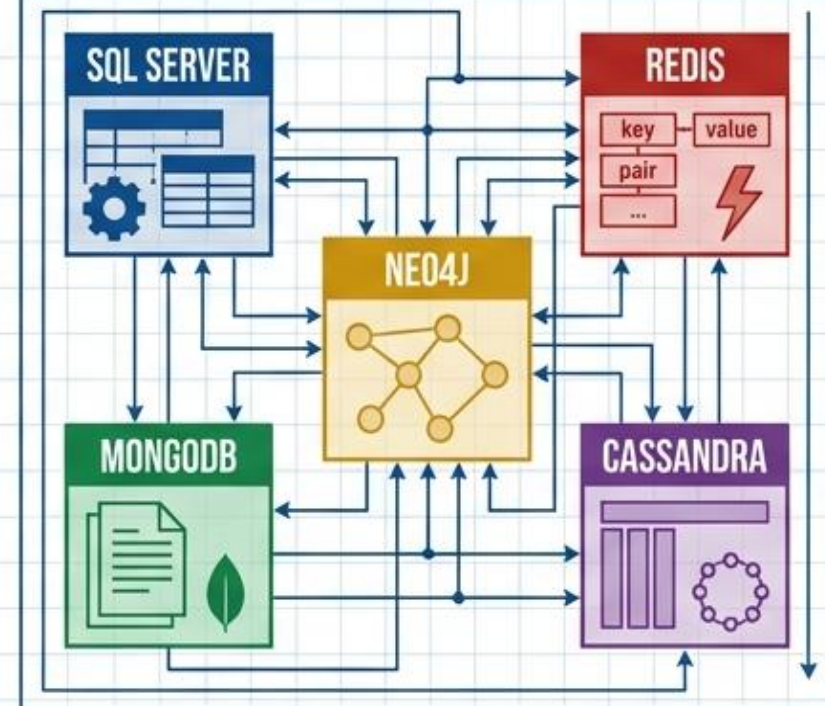


Flujo puntual de reserva de turnos

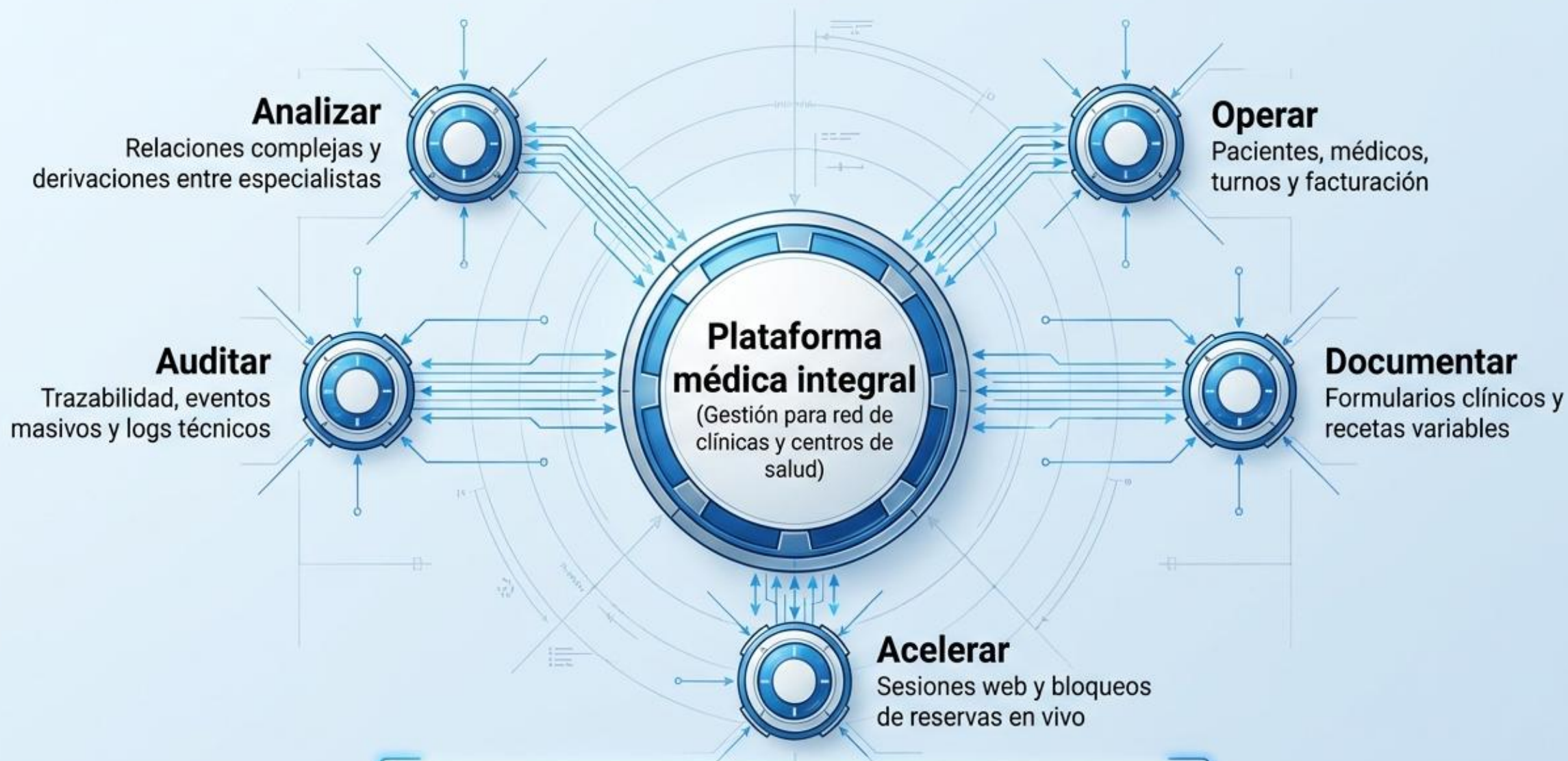
CLASE 11: ARQUITECTURA EMPRESARIAL HÍBRIDA



SQL SERVER + MONGODB + REDIS + CASSANDRA + NEO4J



Caso conductor: red médica con múltiples necesidades de datos



Una sola base de datos difícilmente resuelve bien todas estas necesidades al mismo tiempo.

El límite de pensar todo con una única base de datos

El diseño profesional comienza por entender la naturaleza del dato y el patrón de consulta.

Dato	Necesidad técnica	Problema si se usa una sola tecnología
Factura médica	Integridad transaccional estricta	Si se guarda sin control, la organización sufre inconsistencias contables.
Formulario clínico	Flexibilidad por especialidad	Si se fuerza a un modelo rígido relacional, genera tablas artificiales y esquemas rotos.
Sesión web / Bloqueo	Velocidad (milisegundos)	Si se consulta siempre en disco, aumenta drásticamente la latencia.
Evento de auditoría	Escritura masiva sin bloqueos	Si se guarda todo en el sistema transaccional maestro, degrada la operación diaria.
Relación médico-paciente	Descubrimiento de vínculos	Si se analiza solo con JOINS relacionales complejos, el sistema colapsa buscando patrones.

Persistencia polimórfica: elegir el motor según el problema

No se trata de usar muchas bases por moda, sino de asignar responsabilidades de persistencia de forma estratégica.



Arquitectura híbrida de datos: integración con propósito

Una arquitectura híbrida bien diseñada reduce tensiones técnicas; una mal diseñada multiplica problemas.

Naturaleza del Dato

¿Qué dato se guarda exactamente?

¿Cuál motor actuará como la fuente de verdad definitiva?

¿Qué nivel de consistencia (inmediata vs. eventual) requiere cada operación?

Operación y Riesgos

¿Quién lo consulta, con qué patrón y con qué frecuencia?

¿Qué latencia y volumen de datos se esperan?

¿Qué ocurre si un componente de persistencia falla o queda desactualizado?

Arquitectura propuesta para la red médica



Python coordina; las bases de datos persisten según según su naturaleza.



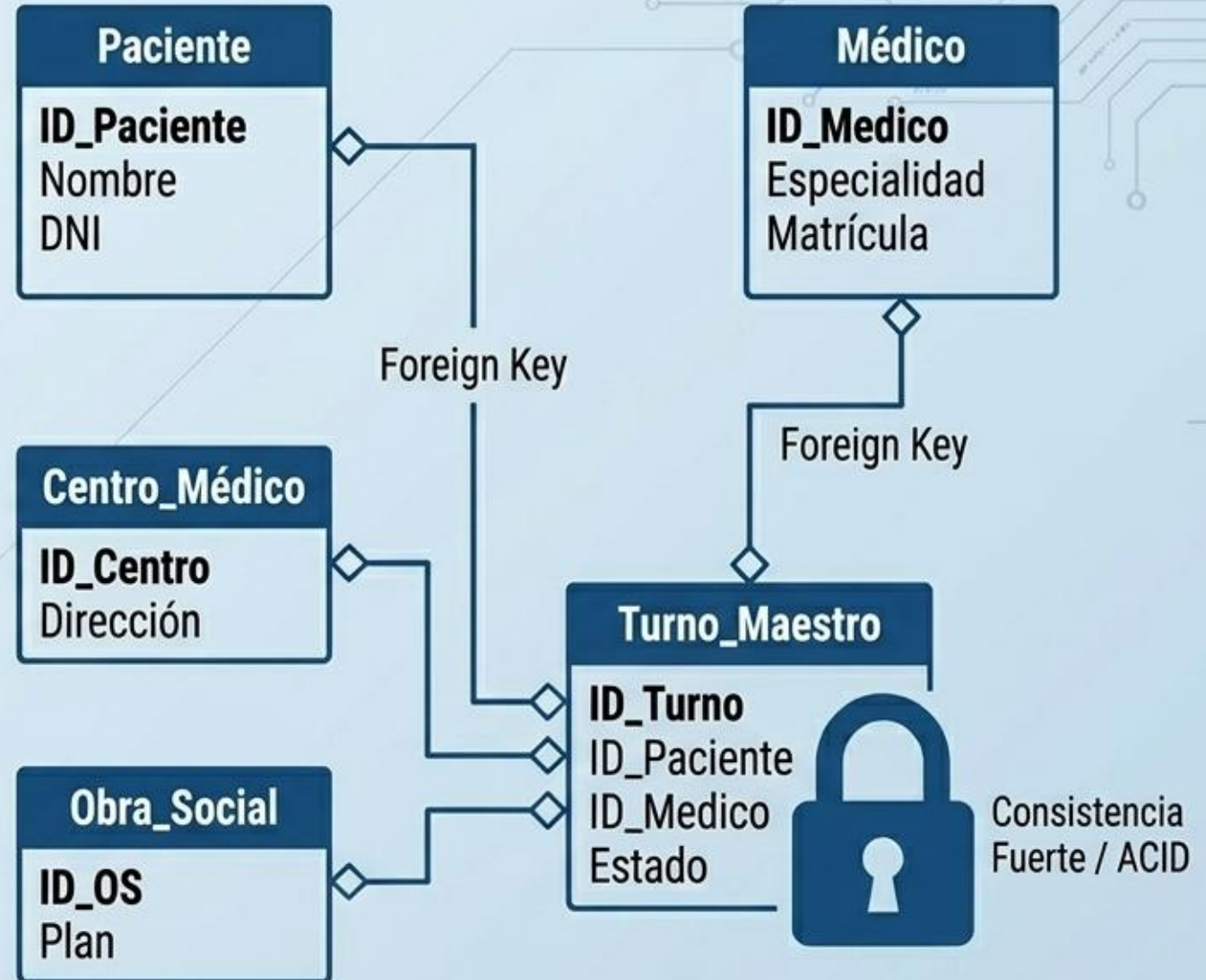
SQL Server: el núcleo estructurado y transaccional



Utilizado para la **Fuente de Verdad** principal.

Asegura que los datos maestros y transacciones críticas (como facturación) jamás queden huérfanos o inconsistentes.

SQL Server sostiene el núcleo confiable del negocio.



MongoDB para reservas, formularios y documentos clínicos

Ideal para datos **operativos semiestructurados**. Una consulta cardiológica, dermatológica o traumatológica requieren campos diferentes que evolucionan con el tiempo.

La flexibilidad documental permite almacenar estructuras clínicas variables sin romper esquemas rígidos.



```
{
  "_id": "doc_84729",
  "tipo": "Formulario Cardiológico",
  "paciente_id": 1042,
  "mediciones_vitales": {
    "presion": "120/80",
    "ritmo_cardiaco": 72
  },
  "notas_medico": "Paciente estable. Requiere holter."
}
```

Formulario Dermatológico

```
{
  "_id": "doc_84730",
  "tipo": "Formulario Dermatológico",
  "paciente_id": 1043,
  "lesiones_piel": {
    "ubicacion": "Antebrazo",
    "tamano_mm": 15
  },
  "notas_medico": "Seguimiento de nevus."
}
```

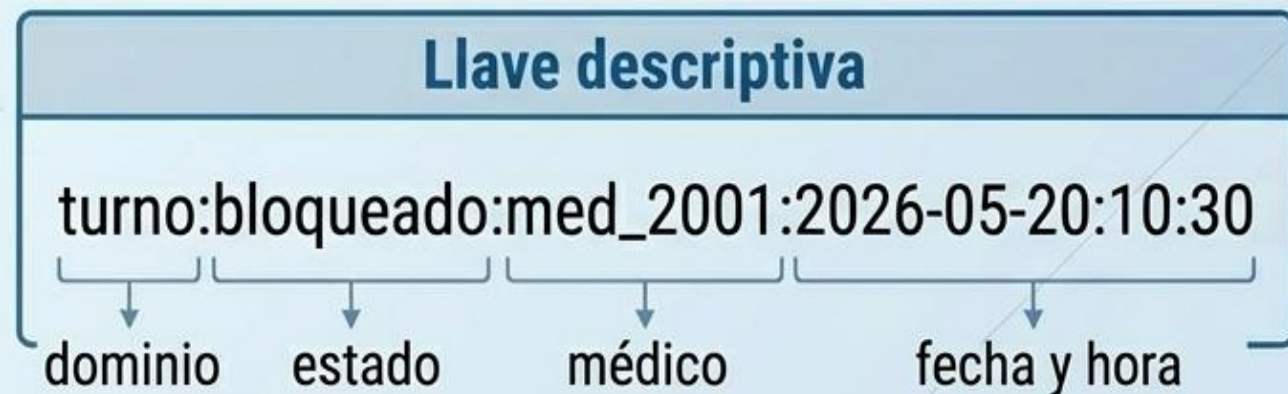

Redis: velocidad en memoria para estados y bloqueos

Almacena sesiones, cachés y bloqueos en memoria RAM.
Evita conflictos de concurrencia cuando múltiples
pacientes intentan reservar el mismo turno al mismo tiempo.



TTL: 15:00 min

(el dato se auto-elimina si
la reserva no se completa)



Valor

```
{  
  "paciente_temp": 8842  
}
```

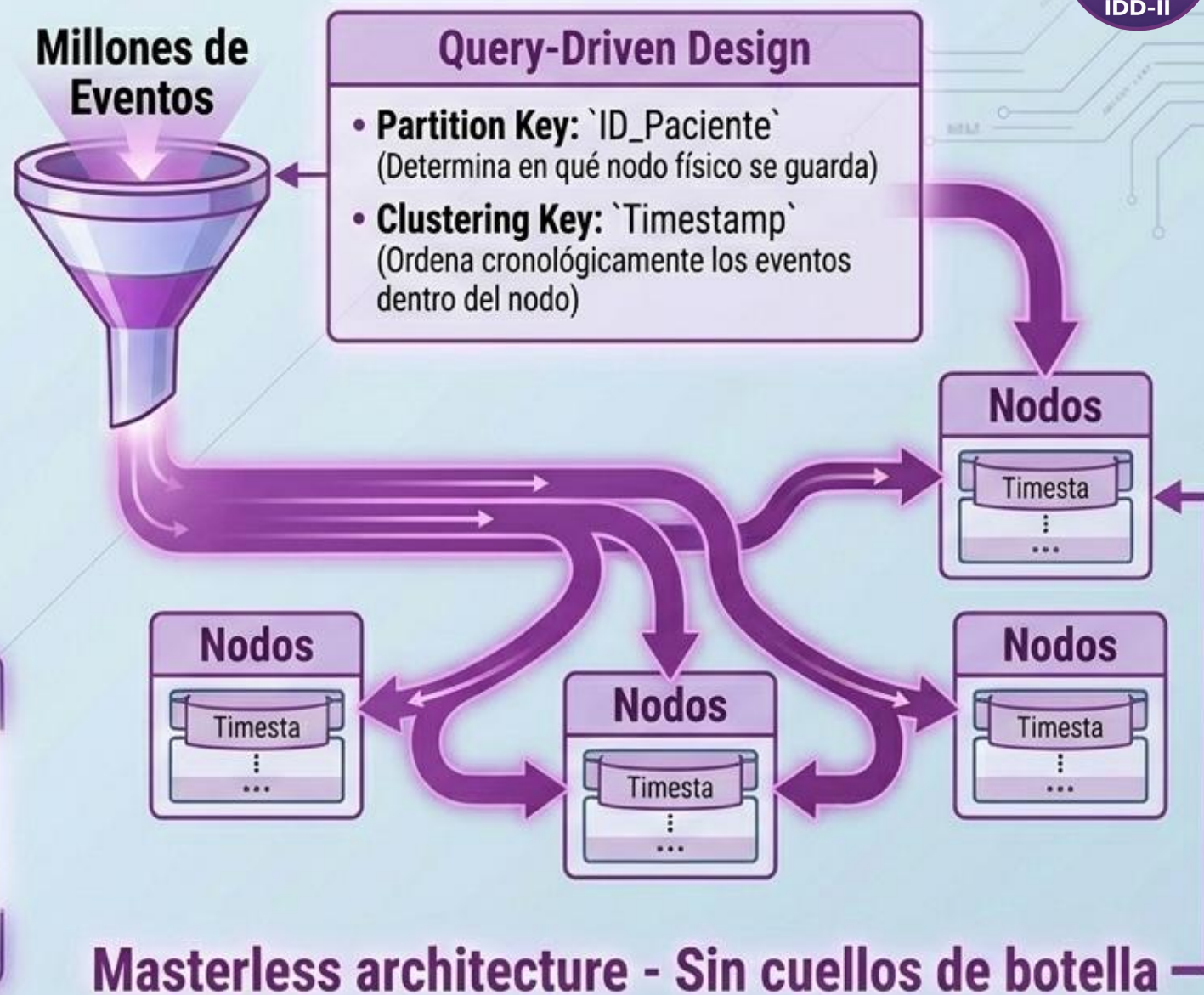
**Redis aporta velocidad extrema y estados temporales sin
penalizar los discos de las bases transaccionales.**

Cassandra: eventos masivos y trazabilidad



El diseño parte estrictamente de la consulta esperada. Se utiliza para registrar cada acción operativa, garantizando trazabilidad y auditoría inmutable a alta velocidad.

Cassandra brilla cuando el diseño se orienta a la consulta y se requiere ingesta masiva sin cuellos de botella.



Neo4j: análisis de vinculación y grafos



Optimizado para recorrer relaciones de muchos saltos sin utilizar costosos JOINS. Permite descubrir patrones ocultos de derivación, fraudes de cobertura médica o clusters de atención.



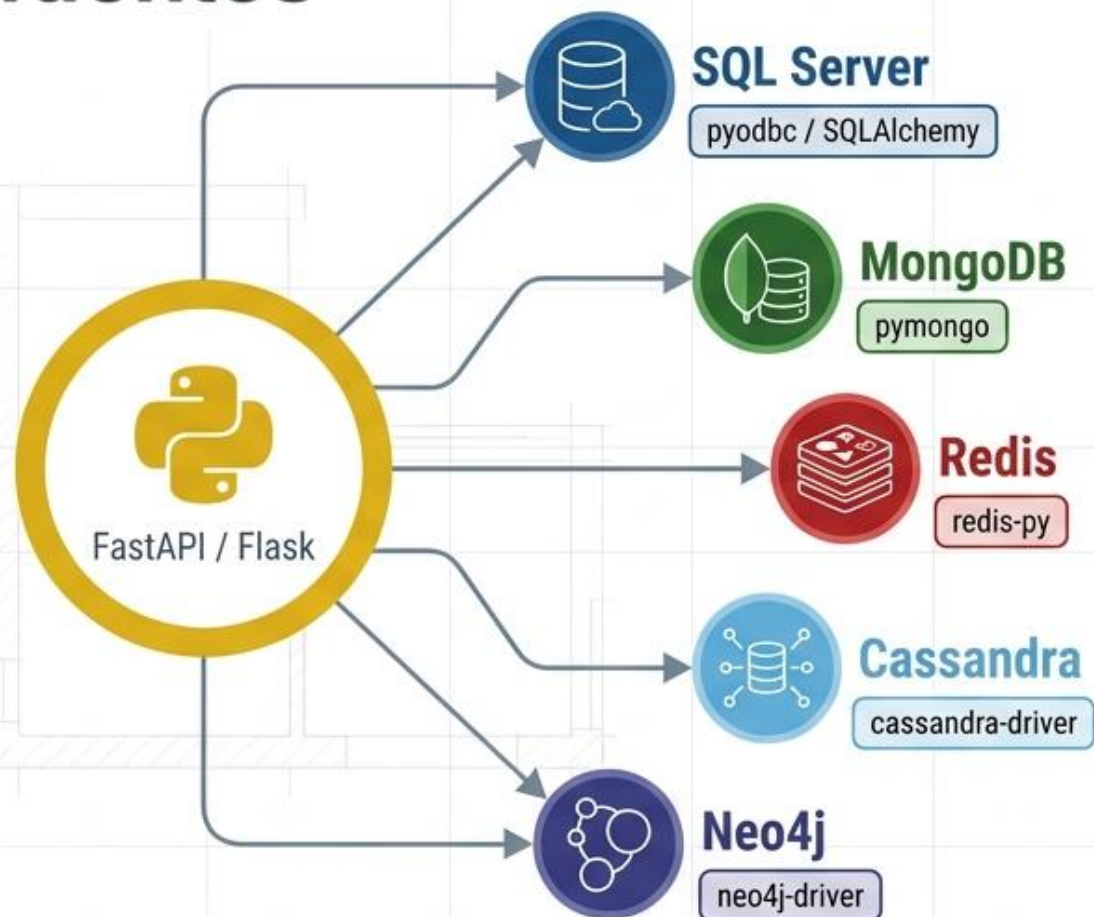
El grafo se justifica cuando el análisis de relaciones es central para el problema.

Matriz de decisión: qué motor usar y por qué

La arquitectura híbrida se decide por responsabilidad, no por moda tecnológica.

Motor	Modelo	Tipo de Dato	Responsabilidad	Consulta Típica	Riesgo si se usa mal
SQL Server	Relacional	Estructurado	Fuente de verdad / Transacciones	<code>`SELECT * WHERE ID=x`</code>	Rigidez ante cambios de esquemas operativos.
MongoDB	Documental	JSON / Jerárquico	Flexibilidad de formularios	<code>`db.forms.find({"tipo": "x"})`</code>	Desorden si se usa como repositorio sin reglas.
Redis	Clave-Valor	Efímero / Memoria	Bloqueos / Caché / Sesiones	<code>`GET turno:bloqueado:1`</code>	Pérdida de datos si se usa como base definitiva.
Cassandra	Wide-column	Series de tiempo	Eventos masivos / Trazabilidad	<code>`SELECT * WHERE partition_key=x`</code>	Ineficiencia total si se consulta por campos sin índice.
Neo4j	Grafo	Conexiones	Análisis de vínculos	<code>`MATCH (p)-[:DERIVA]->(m)`</code>	Sobrecarga si se usa solo para guardar datos aislados.

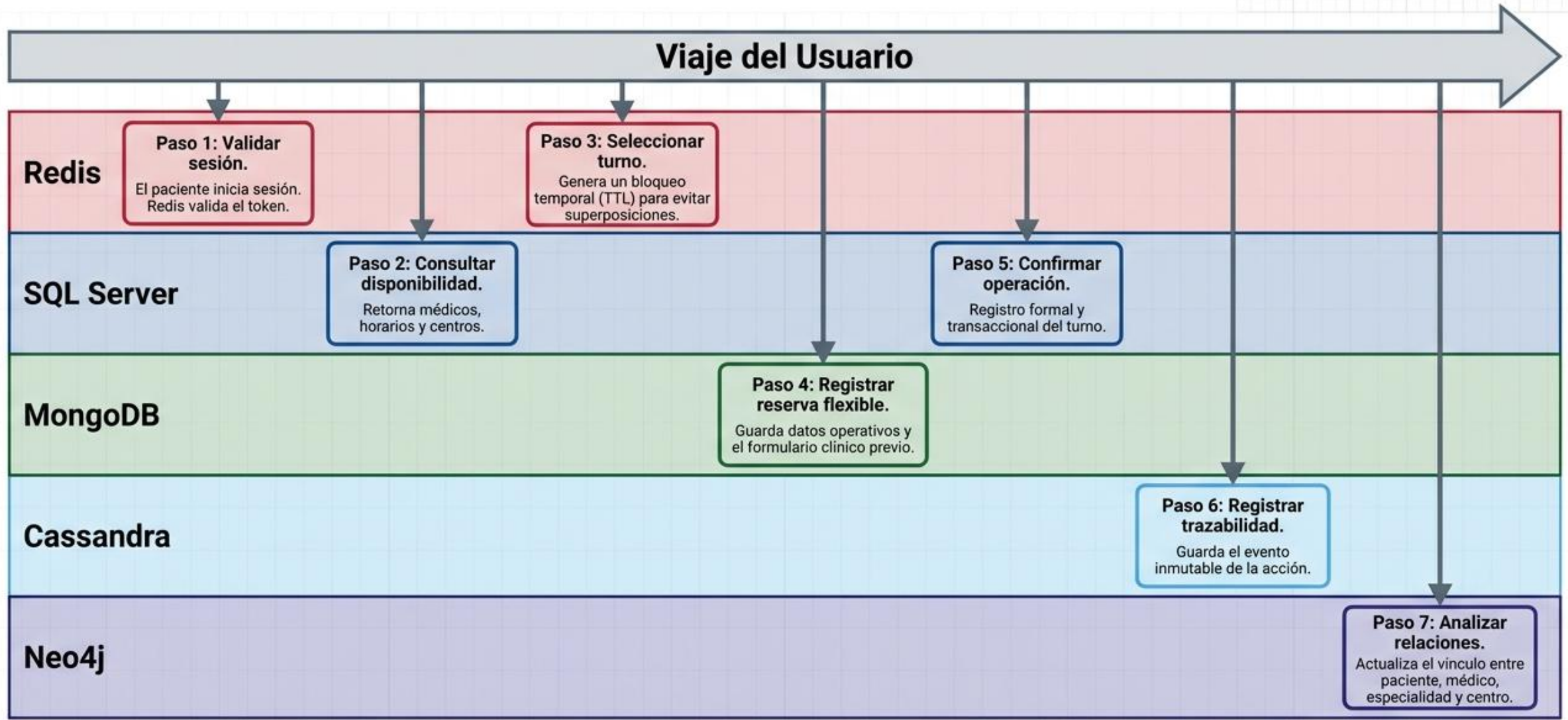
Python como coordinador de acceso a múltiples fuentes



```
1  def procesar_reserva(request):
2
3
4      validar_usuario(request.usuario)
5      bloquear_turno_en_redis(request.turno)
6      consultar_datos_maestros_sql(...)
7      guardar_reserva_mongodb(request)
8      registrar_evento_cassandra(request)
9      actualizar_relacion_neo4j(request)
10     return {'estado': 'reserva_iniciada'}
```

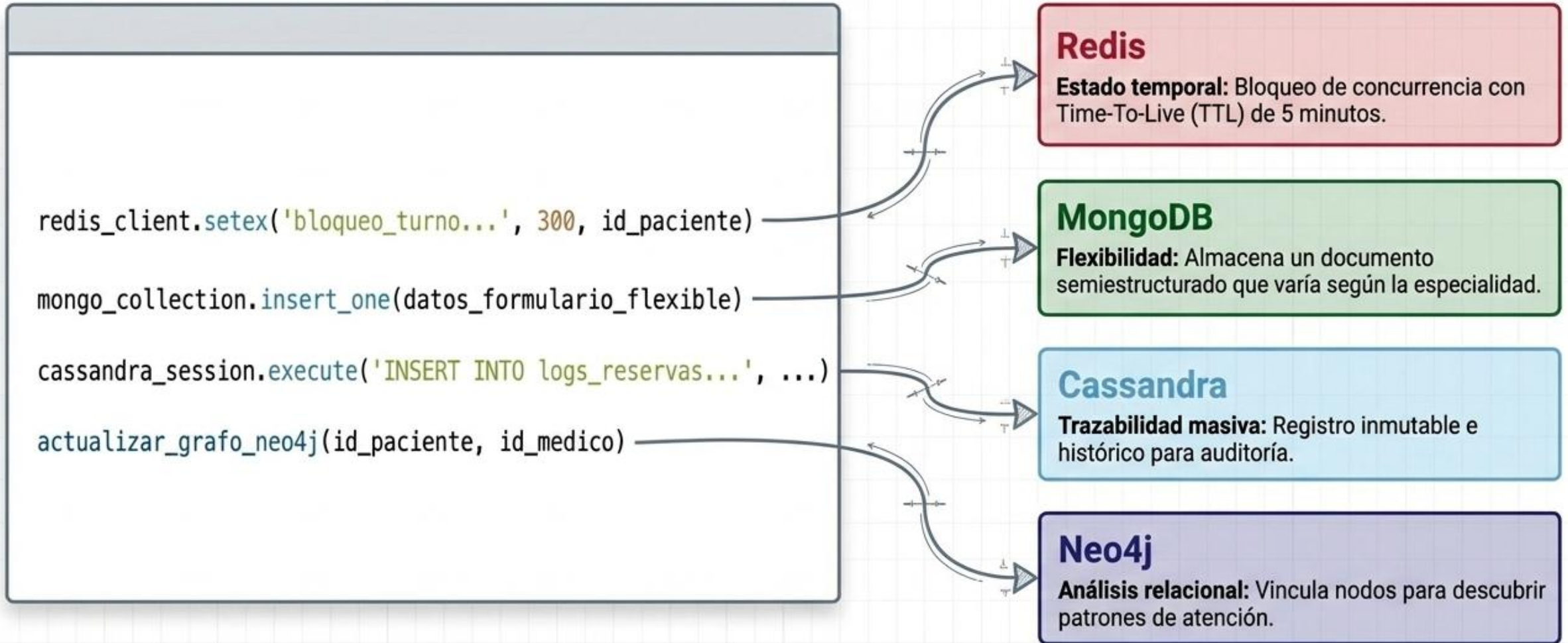
Mensaje Clave: Python coordina el flujo, pero la arquitectura define qué responsabilidad tiene cada motor.

Architectural Blueprint for Data



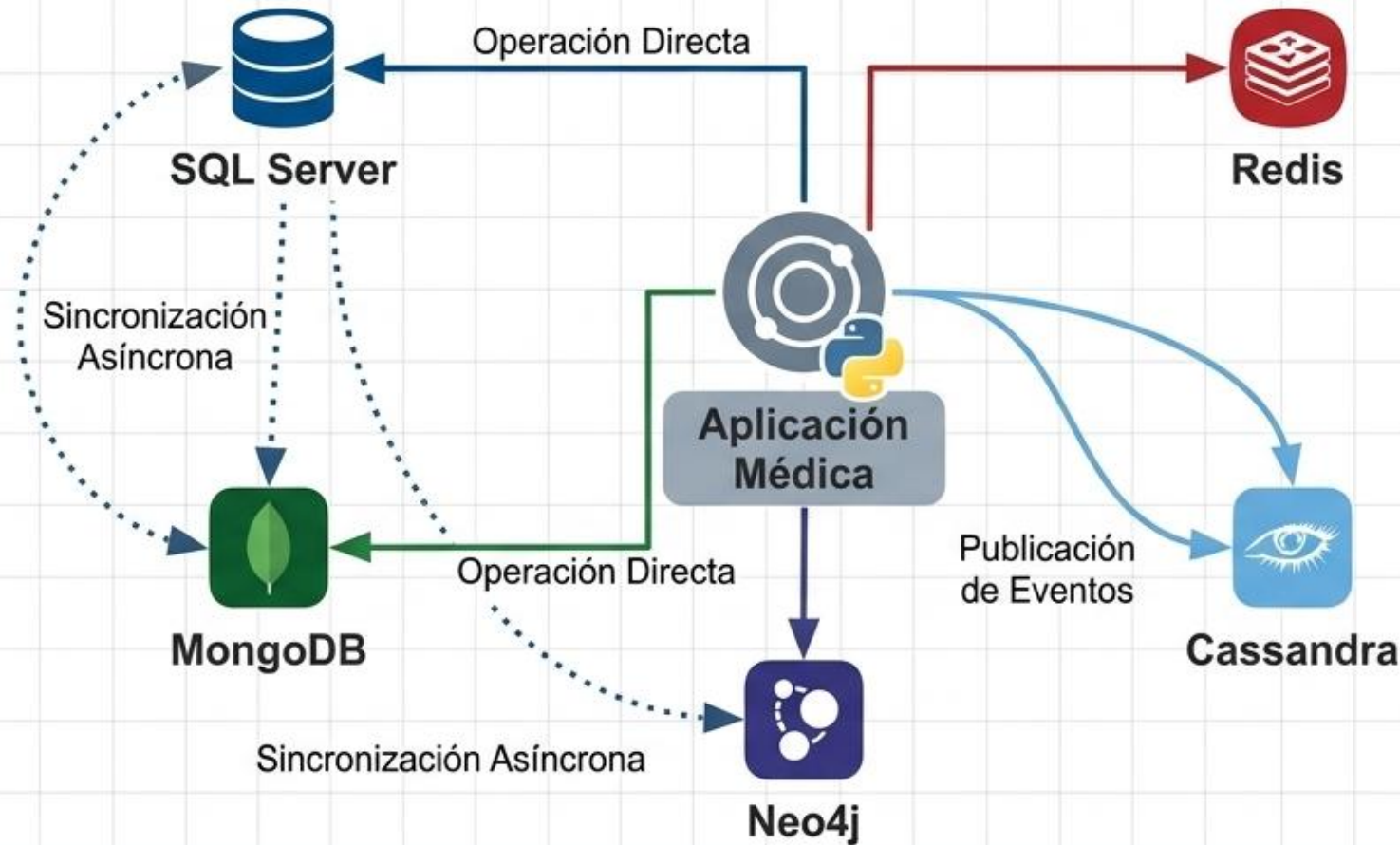
Mensaje Clave: Una operación aparentemente simple puede involucrar múltiples responsabilidades de datos.

Ejemplo de orquestación básica con Python



Mensaje Clave: El código no debe ocultar el diseño; cada instrucción responde a una decisión arquitectónica.

Persistencia e integración: cómo circulan los datos



Reglas de Arquitectura

- **Sincronización:** Definir qué se replica y cada cuánto tiempo (batch vs. streaming).
- **Tolerancia a fallos:** Qué ocurre con la aplicación si un motor auxiliar se cae (degrada graciosamente vs. bloquea la transacción).

Mensaje Clave: Una arquitectura híbrida requiere reglas de integración, no solo conexiones.

Fuente de verdad: evitar contradicciones entre motores



Duplicar puede ser correcto, pero duplicar sin gobierno es riesgoso

Niveles de consistencia: de inmediata a eventual



Mensaje Clave: Diseñar consistencia es decidir qué debe ser exacto ahora y qué puede actualizarse después.

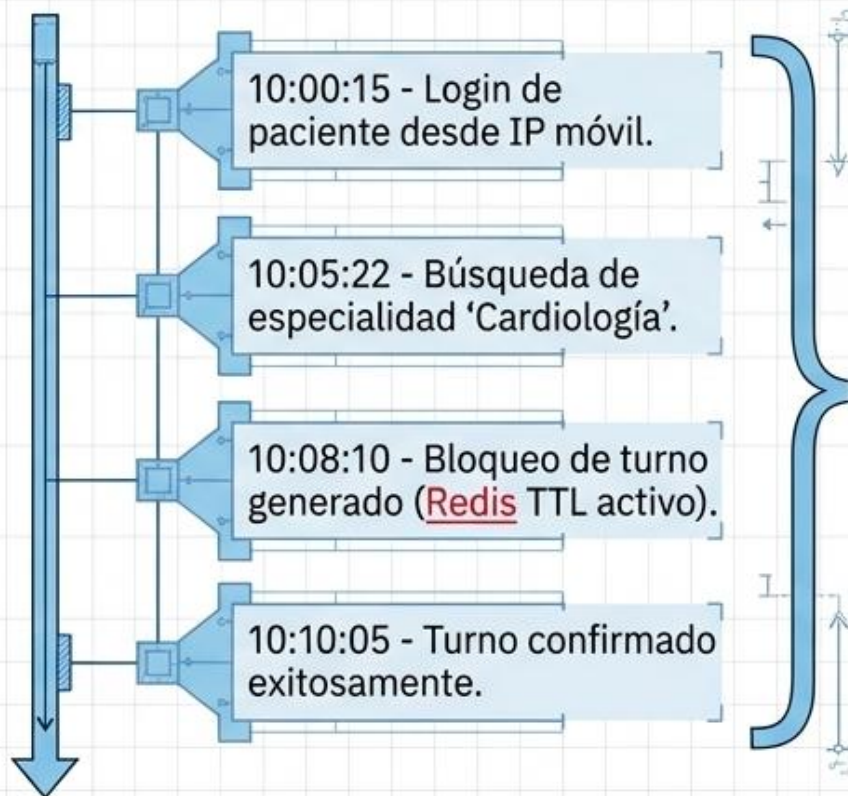
Trazabilidad: reconstruir qué ocurrió y por qué

Estado Actual - SQL Server



(No hay contexto previo)

Historial de Eventos - Cassandra



Auditoría completa:

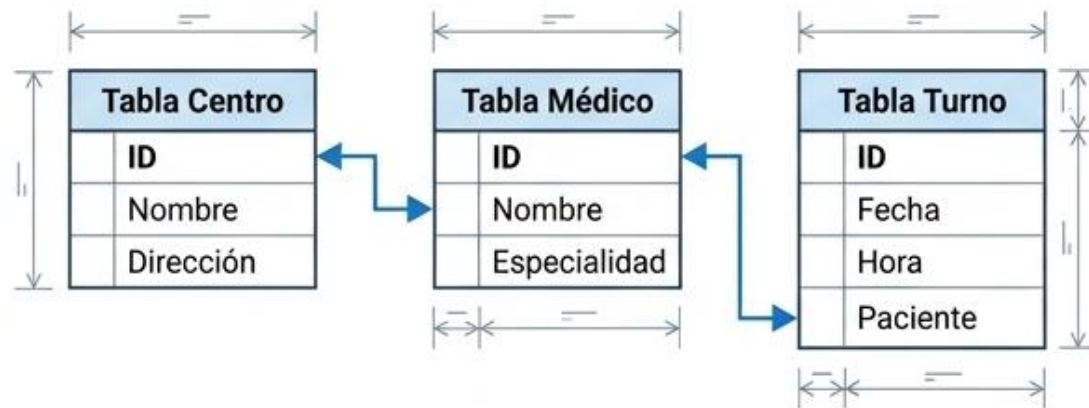
Quién, cuándo, desde dónde, qué dato y con qué resultado

Mensaje Clave: La trazabilidad convierte operaciones aisladas en historia verificable.

Diseñar desde las preguntas que el sistema debe responder

Diseño Relacional (Ej. SQL Server)

Enfoque: Entidades Normalizadas.



El diseño busca no repetir datos.

Diseño Orientado a Consultas (Ej. Cassandra / NoSQL)

Enfoque: Diseño por Pregunta del Negocio.

¿Cuáles son los turnos por centro médico hoy?

Tabla turnos_por_centro_dia

Contiene toda la información necesaria pre-agrupada para responder en una sola lectura sin JOINS.

Mensaje Clave: Una buena arquitectura responde preguntas concretas, no solo almacena datos.

Errores que debilitan una arquitectura polígloa

	Riesgo	Consecuencia	Control
⚠	No definir una fuente de verdad.	Datos contradictorios entre motores.	Implementar matriz de autoridad por entidad.
⚠	Usar Redis como base definitiva.	Pérdida de datos críticos por volatilidad (memoria).	Limitar su uso a caché, sesiones y TTL.
⚠	Modelar Cassandra como si fuera SQL.	Degradación extrema de latencia por escaneos completos.	Modelado estricto orientado a consultas.
⚠	Adoptar múltiples bases "por moda".	Complejidad operativa insostenible.	Exigir justificación arquitectónica para cada motor.

Mensaje Clave: La complejidad híbrida se justifica solo si está gobernada.

Cómo evaluar si la solución híbrida funciona



Métricas Técnicas

- Latencia y Throughput.
- Tiempos de respuesta bajo carga concurrente.



Métricas Funcionales

- Consistencia lograda (¿El usuario ve el dato correcto a tiempo?).
- Capacidad de respuesta de consultas complejas.



Métricas de Negocio

- Disponibilidad general del sistema frente a fallos parciales.
- Nivel de trazabilidad auditable (seguridad y gobierno).

El diseño híbrido debe demostrar valor operativo, analítico y técnico.

Cómo llevar este enfoque al Trabajo Práctico



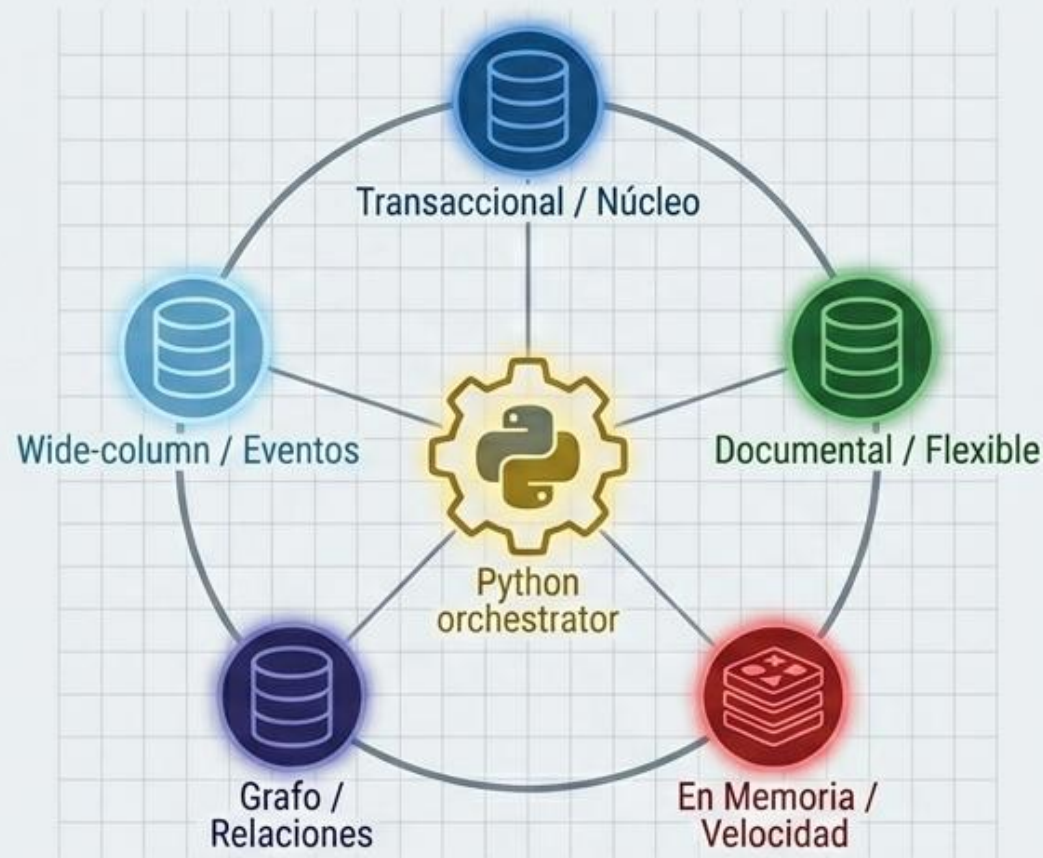
Cheat Sheet



Dato	Motor Elegido	Motivo	Consulta Principal	Consistencia	Riesgo	Control
Turno confirmado	SQL Server	Operación crítica del negocio	Estado actual del turno por ID	Alta / Inmediata	Doble reserva concurrente	Bloqueo previo en Redis + Transacción ACID en SQL

Una propuesta profesional no dice solo “qué tecnología usa”, sino “por qué la usa y qué problema resuelve”.

Una arquitectura híbrida no acumula bases: organiza responsabilidades



“El valor de una arquitectura híbrida no está en la cantidad de tecnologías utilizadas, sino en la claridad con la que cada una resuelve una responsabilidad de datos.”

Diseñar datos es decidir responsabilidades, no solo elegir motores.



Práctica de laboratorio:

Gestión de turnos médicos en una arquitectura híbrida de persistencia polígloa

Este laboratorio tiene como propósito diseñar una arquitectura de datos híbrida para una plataforma integral de gestión médica, integrando múltiples motores de datos según su responsabilidad funcional. La actividad busca que cada grupo comprenda que una solución moderna no se construye acumulando bases de datos, sino asignando correctamente qué tipo de información debe vivir en cada motor, qué consulta debe responder, qué nivel de consistencia necesita y cómo se integra con el resto del sistema.

La práctica debe permitir analizar cómo Python puede coordinar el flujo de reserva, confirmación, trazabilidad y análisis, conectándose conceptualmente con SQL Server, MongoDB, Redis, Cassandra y Neo4j.



PROXIMA CLASE



Clase 12 - Big Data y visión sistémica de la ingeniería de datos

Tema:

Manejo de grandes volúmenes de datos.

Big Data: concepto y contexto.

Hadoop como ecosistema (HDFS, procesamiento distribuido).
